

Arquitetura e Organização do BIP II

Prof. Roberto de Matos

roberto.matos@ifsc.edu.br



**INSTITUTO
FEDERAL**
Santa Catarina

Câmpus
São José

- Estender a arquitetura do BIP para incluir instruções de desvio condicional e incondicional, criando o BIP II
- Propor uma organização para BIP II
- Desenvolver o decodificador do BIP II



- Estender a arquitetura do BIP para incluir instruções de desvio condicional e incondicional, criando o BIP II
- Propor uma organização para BIP II
- Desenvolver o decodificador do BIP II

Material adaptado do Prof. Dr. Cesar Zeferino (Univali).



Arquitetura BIP II

- Inclui um registrador com dois bits de estado:
 - **STATUS.Z (Zero)**: Resultado da última operação aritmética = 0
 - **STATUS.N (Negative)**: Resultado da última operação aritmética < 0
- Inclui duas classes adicionais de instrução:
 - **Desvio incondicional**: Muda o fluxo de execução do programa, atualizando o PC com o endereço de instrução especificado
 - **Desvio condicional**: Se a condição especificada for verdadeira, desvia para o endereço de instrução especificado



■ Conjunto de instruções estendido:

Instrução de desvio incondicional

JMP

Jump

Desvia incondicionalmente



■ Conjunto de instruções estendido:

Instrução de desvio incondicional

JMP	Jump	Desvia incondicionalmente
------------	------	---------------------------

Instruções de desvio condicional

BEQ	Branch on Equal	Desvia se igual
BNE	Branch on Not Equal	Desvia se não igual
BGT	Branch on Greater Than	Desvia se maior que
BGE	Branch on Greater or Equal	Desvia se maior ou igual que
BLT	Branch on Less Than	Desvia se menor que
BLE	Branch on Less or Equal	Desvia se menor ou igual que



■ Conjunto de instruções estendido:

Instrução de desvio incondicional

JMP	Jump	Desvia incondicionalmente
------------	------	---------------------------

Instruções de desvio condicional

BEQ	Branch on Equal	Desvia se igual
BNE	Branch on Not Equal	Desvia se não igual
BGT	Branch on Greater Than	Desvia se maior que
BGE	Branch on Greater or Equal	Desvia se maior ou igual que
BLT	Branch on Less Than	Desvia se menor que
BLE	Branch on Less or Equal	Desvia se menor ou igual que

- Instruções de desvio condicional devem ser precedidas por uma operação SUB ou SUBI sobre os operandos a serem comparados.

■ Exemplo:

```
1 LD    A
2 SUBI  2
3 BLE  addr    ; PC=addr se (A) <= 2
```



Arquitetura estendida do BIP - BIP II

Cód. operação	Instrução	Operação
01000	BEQ endereço	Se (STATUS.Z=1) então PC ← endereço Se não PC ← PC + 1
01001	BNE endereço	Se (STATUS.Z=0) então PC ← endereço Se não PC ← PC + 1
01010	BGT endereço	Se (STATUS.Z=0) e (STATUS.N=0) então PC ← endereço Se não PC ← PC + 1
01011	BGE endereço	Se (STATUS.N=0) então PC ← endereço Se não PC ← PC + 1
01100	BLT endereço	Se (STATUS.N=1) então PC ← endereço Se não PC ← PC + 1
01101	BLE endereço	Se (STATUS.Z=1) ou (STATUS.N=1) então PC ← endereço Se não PC ← PC + 1
01110	JMP endereço	PC ← endereço
01111 - 11111	Reservados para as futuras gerações	



Abstração	Código em C	Código na ling. de montagem
Teste de condição tipo if-then	<pre>if (A==B) { // Bloco 1 } // Bloco 2</pre>	<pre>LD A ; ACC ← A SUB B ; ACC ← ACC - B BNE L1 ... ; Bloco 1 L1: ... ; Bloco 2</pre>
Teste de condição tipo if-then-else	<pre>if (A==B) { // Bloco 1 } else { // Bloco 2 } // Bloco 3</pre>	<pre>LD A ; ACC ← A SUB B ; ACC ← ACC - B BNE L1 ... ; Bloco 1 JMP L2 L1: ... ; Bloco 2 L2: ... ; Bloco 3</pre>



Abstração	Código em C	Código na ling. de montagem
Laço de repetição do tipo while	<pre>i = 0; while (i<10) { // Bloco 1 i++; } // Bloco 2</pre>	<pre>LDI 0 ; ACC ← 0 STO I ; I ← ACC L1: SUBI 10 ; ACC ← ACC - 10 BGE L2 ... ; Bloco 1 LD I ; ACC ← I ADDI 1 ; ACC ← ACC + 1 STO I ; I ← ACC JMP L1 L2: ... ; Bloco 2</pre>
Laço de repetição do tipo for	<pre>for(i=0;i<10;i++){ // Bloco 1 } // Bloco 2</pre>	Igual ao laço while



Abstração	Código em C	Código na ling. de montagem
Laço de repetição do tipo do-while	<pre>i = 0; do { // Bloco 1 i++; } while (i<10) // Bloco 2</pre>	<pre>LDI 0 ; ACC ← 0 STO I ; I ← ACC L1: ... ; Bloco 1 LD I ; ACC ← I ADDI 1 ; ACC ← ACC + 1 STO I ; I ← ACC SUBI 10 ; ACC ← ACC - 10 BLT L1 ... ; Bloco 2</pre>



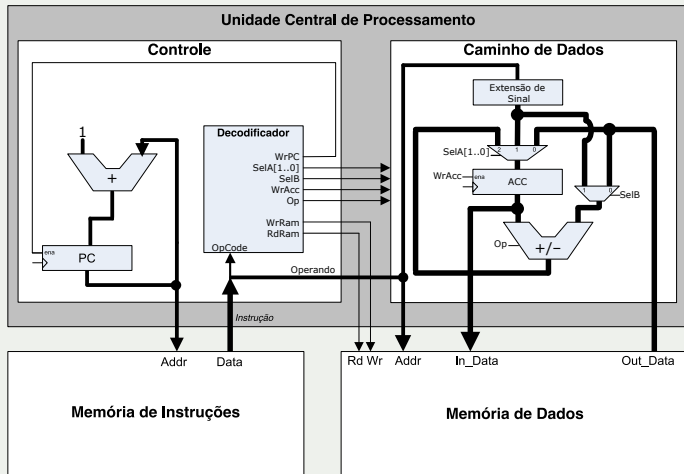
Organização BIP II

- Bits de estado da Unidade Aritmética (UA):
 - STATUS.Z (**Zero**): Resultado da última operação aritmética = 0
 - STATUS.N (**Negative**): Resultado da última operação aritmética < 0
- Modificar a organização.
- Projeto do novo decodificador.

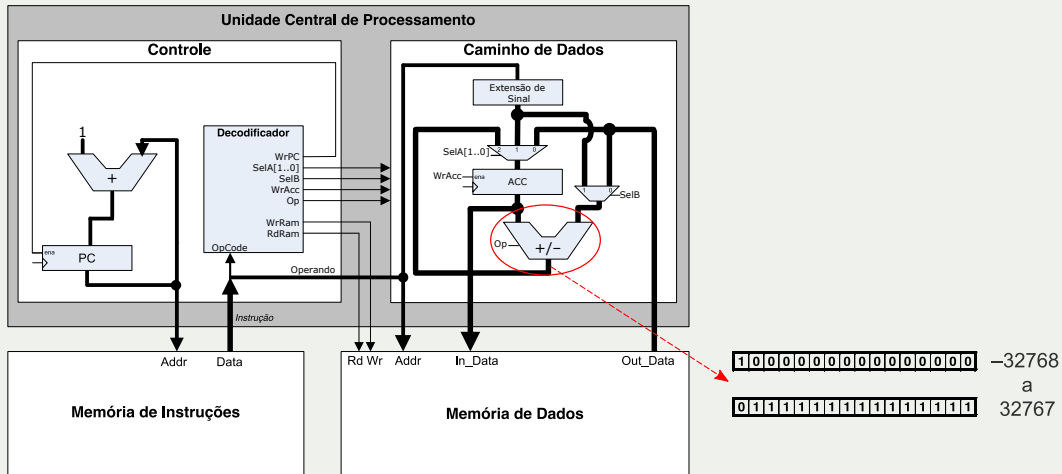


Bits de estado na UA

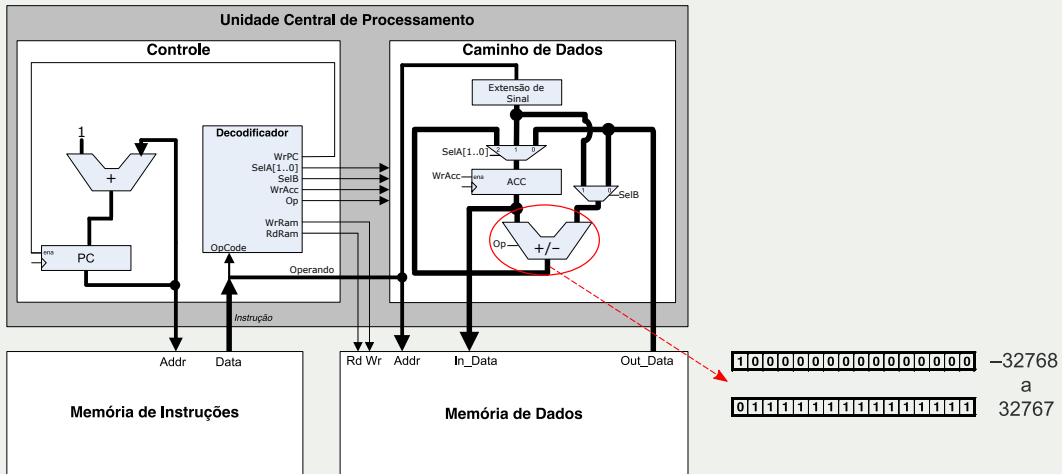
■ STATUS.N:



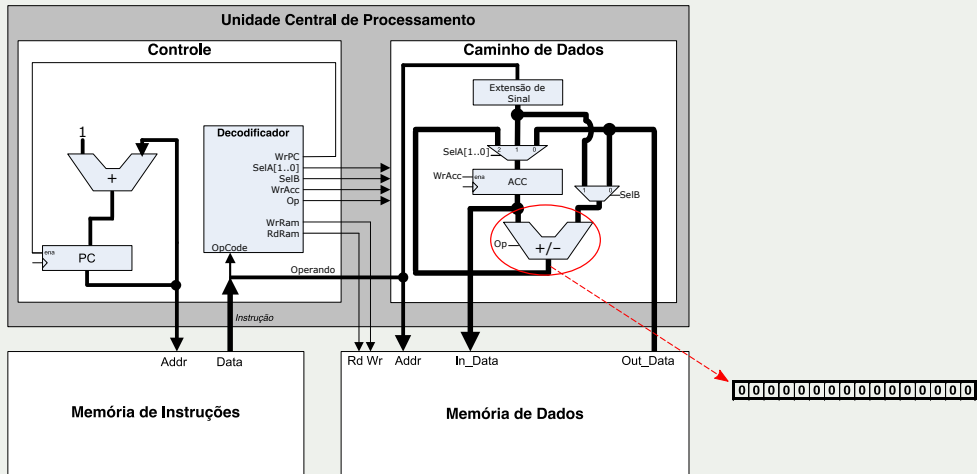
■ STATUS.N:



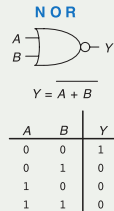
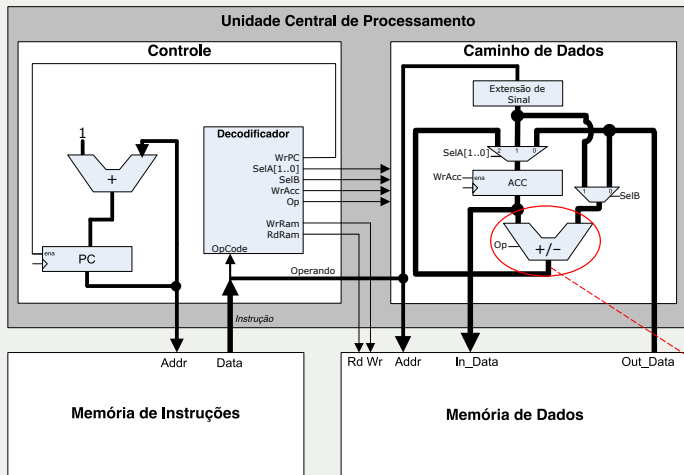
■ STATUS.N: **BIT 15** da saída UA



■ STATUS.Z:



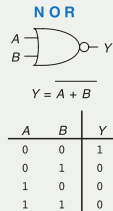
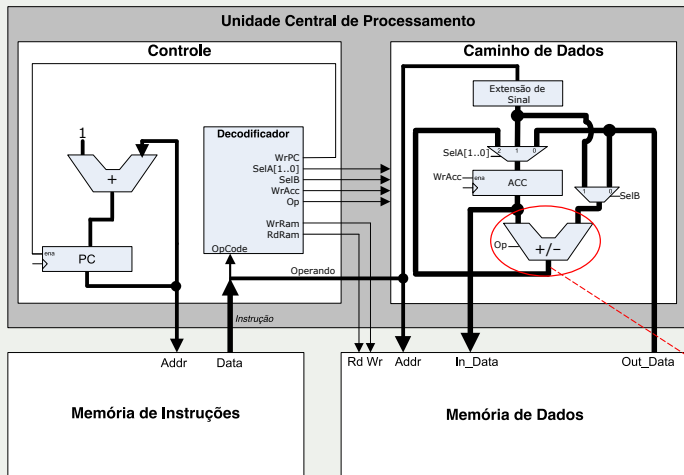
■ STATUS.Z:



0000000000000000



■ STATUS.Z: Porta NOR de 16 entradas

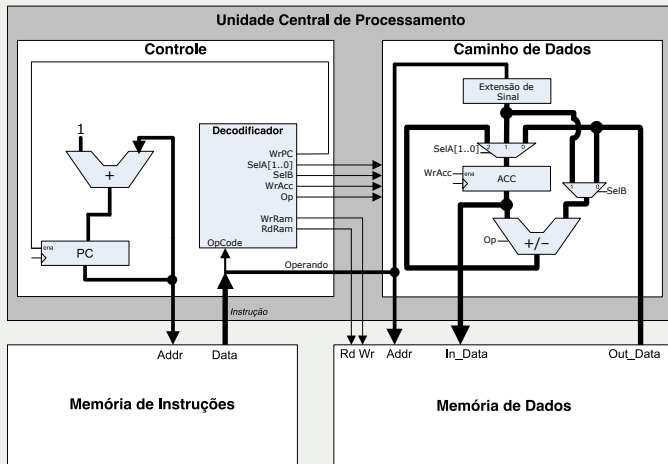


0000000000000000

Modificar a organização

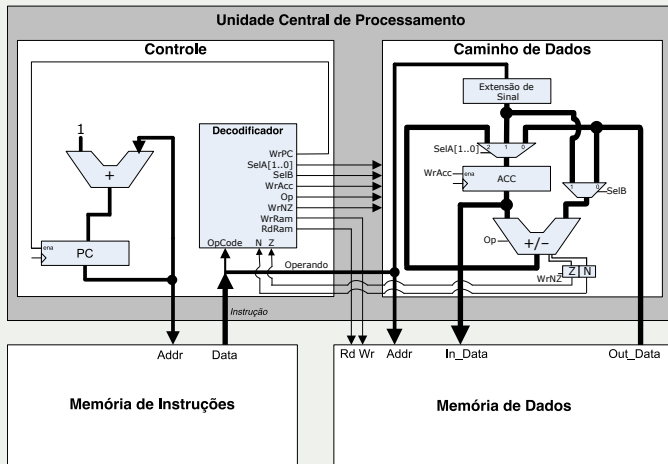
Modificar a organização

- Registrador STATUS.x e ligação ao decodificador.



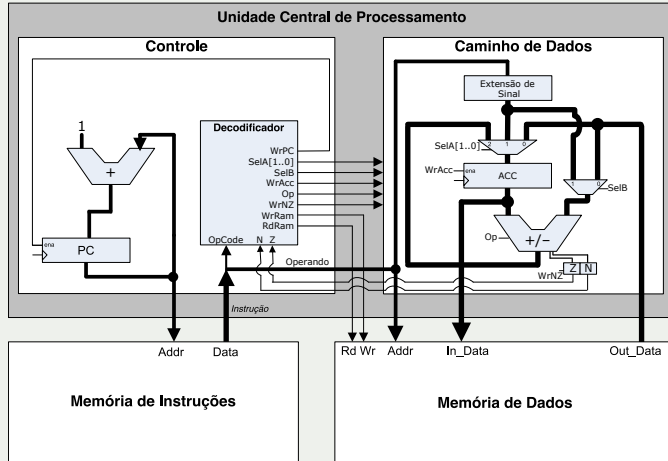
Modificar a organização

- Registrador STATUS.x e ligação ao decodificador.



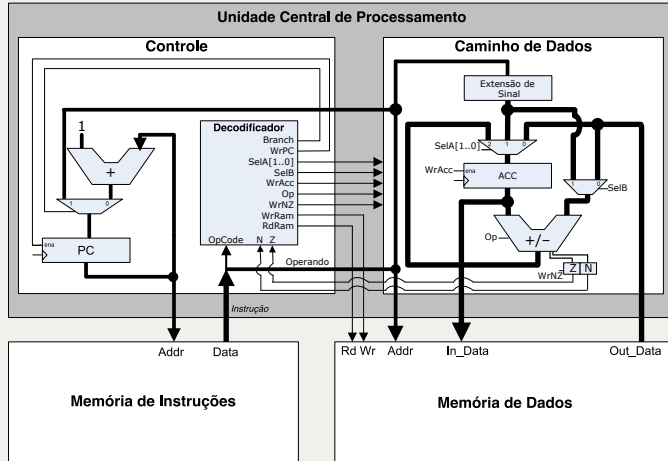
Modificar a organização

■ Controle do desvio.



Modificar a organização

■ Controle do desvio.



Projeto do novo decodificador

- Adicionar novas instruções.
- A partir do `OpCode` das instruções aritméticas, gerar o sinal `WrNZ`.
- A partir dos sinais de `STATUS.x` e `OpCode` dos desvios, gerar sinal `Branch`.



FIM!